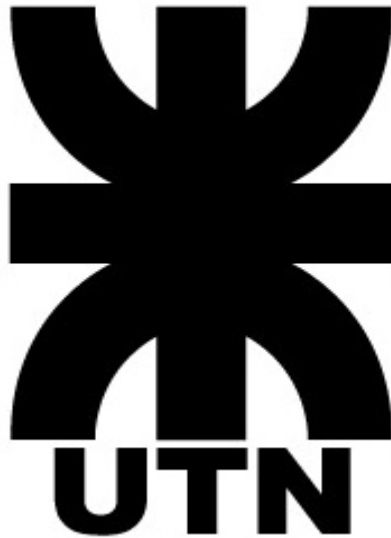




Tecnicatura universitaria en programación a distancia

Materia: Programación II

Alumno: Matías Facundo Herrera, Andrés Oscar Bonelli, Facundo Alejandro Butti, Tomás José Bufo

Trabajo práctico Integrador Final

Aplicación Java con relación 1 → 1 unidireccional + DAO + MySQL

Consignas generales

Desarrollar una aplicación en Java que modele dos clases relacionadas mediante una asociación unidireccional 1 a 1 (la clase "A" referencia a la clase "B"), persistiendo datos en una base relacional mediante JDBC y el patrón DAO, con operaciones transaccionales (commit/rollback) y menú de consola para CRUD.

- Lenguaje: Java (recomendado 21).
- Persistencia: JDBC (sin ORM) con MySQL.
- Patrón de diseño: DAO y capa Service.
- Código limpio, legible, con manejo de excepciones en todas las capas.
- Relación 1→1 unidireccional: sólo la clase "A" contiene el atributo que referencia a "B".
- Equipos de 4 personas, sin excepción

Miembros del equipo y roles

Andrés Oscar Bonelli: Desarrollo de Lógica de negocio y capa de acceso a datos.

Matías Facundo Herrera: Desarrollo de la consola de navegación, manejo de entradas del usuario y testeo de la aplicación.

Facundo Alejandro Butti: Desarrollo de diagrama UML, testeo de aplicación, mejoras y validaciones.

Tomás Jose Bufo: Refactorización y mejora de la lógica de negocio.

Implementación y optimización de validaciones en las entidades y servicios.

Revisión y ajuste del manejo de transacciones (commit/rollback) en la capa Service.

Corrección de estructuras internas y mejoras generales de calidad del código.

Entidades utilizadas

Se optó por el ejemplo proporcionado en el documento de este trabajo dónde entidad A es **empresa** y entidad B es **domicilio fiscal**.

Clase Empresa (A)

Campo	Tipo	Reglas
Id	Long	PK
eliminado	Boolean	Baja lógica
razonSocial	String	NOT NULL, máx 120
cuit	String	NOT NULL, UNIQUE, máx 13
actividadPrincipal	String	máx 80
email	String	máx 120
domicilioFiscal	DomicilioFiscal	Referencia 1 → 1 a B

Clase DomicilioFiscal (B)

Campo	Tipo	Reglas
id	Long	PK
eliminado	Boolean	Baja lógica
calle	String	NOT NULL, máx 100
numero	Integer	NOT NULL, máx 100
ciudad	String	NOT NULL, máx 100

provincia	String	NOT NULL, máx 100
codigoPostal	String	máx 10
pais	String	NOT NULL, máx 100

Se seleccionó estas dos entidades porque representan una perfecta relación unidireccional 1 → 1 donde una empresa posee exactamente un domicilio fiscal, y esa entidad, no puede pertenecer a otra o más empresas. Esto lo hace un perfecto ejemplo para una asociación de estas características.

Además podemos emular un entorno administrativo donde podemos establecer tales comportamientos como: eliminación de baja lógica para las entidades (esto quiere decir que no borramos totalmente los registros de nuestra base de datos), prevención de creación de domicilios los cuales no tengan una compañía asociada, la utilización de transacciones con rollback, entre otros.

Arquitectura por capas y responsabilidad de cada paquete

El proyecto fue diseñado según lo requerido por el informe de una manera clara. La responsabilidad de cada paquete se detalla a continuación.

config/ *Capa de conexión a la base de datos*

Esta capa contiene la clase responsable de la conexión a la base de datos. Centraliza el acceso y configuración a nuestra base de datos. Lo que es principal, aísla la configuración del resto de la aplicación.

entities/ *Capa de entidades*

Esta capa define los objetos de negocio dentro de la aplicación que en este caso son **empresa** y **domicilioFiscal**. Ambas entidades contienen getters y setters y un método toString() para ser mostrado en la consola.

dao/ *Capa de acceso de datos*

Esta capa encapsula todas las conexiones directas con MySQL usando JDBC. Cada DAO utiliza PreparedStatement, cual parametriza las consultas a la base de datos, previniendo inyección SQL y proveyendo de un mejor control de los datos.

service/ *Capa de modelo de negocio*

El paquete servicio actúa como intermediario entre la capa DAO y la consola final del usuario. Utiliza validaciones para obtener datos correctos de las operaciones del usuario. Válida reglas de negocio, garantiza atomicidad y consistencia. Es la capa donde añadimos la lógica para nuestras transacciones.

Persistencia: estructura de la base y transacciones

La persistencia del proyecto se implementó utilizando MySQL como motor de base de datos y JDBC como mecanismo de acceso desde Java. El diseño se centró en garantizar la integridad del modelo 1→1 unidireccional, la consistencia de los datos y la ejecución segura de operaciones transaccionales desde la capa Service.

La base de datos está formada por dos tablas principales: empresa (A) y domicilio_fiscal (B).

La relación 1→1 se implementó mediante una clave foránea única en la tabla B, garantizando que una empresa pueda tener un único domicilio fiscal asociado.

Claves del diseño:

- empresa_id es UNIQUE, garantizando 1 domicilio por empresa.
- El ON DELETE CASCADE asegura integridad en eliminaciones físicas (aunque el sistema usa baja lógica).
- Cada tabla gestiona su propio campo eliminado para implementar la baja lógica, evitando eliminar registros de la base.

La interacción sigue un flujo claro. Cada operación CRUD se realiza mediante PreparedStatement, protegiendo la aplicación contra inyecciones SQL y garantizando un manejo seguro con parámetros.

La capa service gestiona todas las transacciones. Los DAOs aceptan una conexión externa, lo que permite encadenar múltiples operaciones dentro del mismo bloque transaccional. En el inicio de la transacción se setea el autocommit como falso, si todas las operaciones se concretan de manera exitosa se utiliza commit() para finalizar la transacción. De otro modo, se utiliza un rollback y los base de datos queda sin ningún efecto. Al finalizar la transacción, se cierra la conexión con close().

Validaciones y reglas de negocio

Las validaciones y reglas principales están implementadas en las capas de servicio EmpresaService y DomicilioFiscalService y complementadas por restricciones en la base de datos. A continuación se listan las reglas aplicadas en el proyecto:

Reglas de integridad y unicidad:

- cuit es único en la tabla empresa (constraint UNIQUE en el esquema).
- domicilio_fiscal_id en empresa tiene constraint UNIQUE para garantizar 1:1 (un domicilio no puede pertenecer a dos empresas simultáneamente).
- El DAO comprueba en tiempo de operación que el domicilio existe antes de asignarlo a una empresa, si el domicilio pertenece a otra empresa se lanza una excepción.

- Reglas transaccionales y de consistencia:

- Operaciones de insert y update en servicios se ejecutan dentro de transacciones gestionadas manualmente (Connection.setAutoCommit(false) / commit / rollback) para mantener consistencia entre empresa y domicilio_fiscal.
- Al crear una empresa con un domicilio nuevo, el domicilio se inserta primero en la misma transacción y luego la empresa (asegurando atomicidad).
- Al actualizar una empresa que incluye domicilio: si el domicilio trae id se actualiza; si no, se crea primero y luego se asocia.

- Eliminación lógica (soft delete):

- Tanto empresa como domicilio_fiscal usan un campo booleano eliminado para marcar borrados lógicos.
- Las consultas de lectura (DAO) filtran por eliminado = false para ocultar registros borrados.
- La operación eliminar realiza un UPDATE que pone eliminado = true en lugar de borrar físicamente la fila.

- Seguridad y buenas prácticas:

- Se usan PreparedStatement en los DAOs para evitar inyección SQL.
- Excepciones SQL se propaguen y se manejan en los servicios para mostrar mensajes amigables y realizar rollback cuando corresponde.

Pruebas realizadas

A continuación se muestran capturas de pantalla las cuales corresponden a la creación de una empresa y la búsqueda de una empresa a través de su razón social.

```
UTN - TUPaD
Programacion II
Trabajo Final Integrador 2025

=== MENÚ PRINCIPAL ===
1. Crear Empresa
2. Leer empresa por Id
3. Leer empresa por CUIT
4. Buscar empresa por razon social
5. Listar empresas
6. Actualizar empresas
7. Eliminar empresa
8. Crear Domicilio Fiscal
9. Leer domicilio fiscal por Id
10. Listar todos los domicilios fiscales
11. Actualizar domicilio fiscal
12. Eliminar domicilio fiscal
0. Salir
Selecciona una opción: 1
```

Se selecciona la primera opción en el menú por consola.

```
--- Crear Nueva Empresa ---
Razón social: empresa3
CUIT (11 dígitos): 12345678911
Email: empresa3@empresa.com
Actividad principal: empresa

--- Datos del Domicilio Fiscal ---
Calle: falsa
Número: 123
Ciudad: bs as
Provincia: bs as
Código postal (opcional): 1852
País: Argentina
Empresa creada exitosamente con ID: 13
```

La aplicación solicitará los campos necesarios para la creación de la entidad empresa junto con su domicilio fiscal.

	id	eliminado	razon_social	cuit	actividad_principal	email	domicilio_fis
▶	1	0	TechSolutions S.A.	30712345678	Soluciones de Software	contacto@techsolutions.com.ar	1
	2	0	Consultores Asociados S.R.L.	30712345689	Consultoría Empresarial	info@consultores.com.ar	2
	3	0	Constructora del Litoral S.A.	30712345690	Construcción de Obras	obras@constructora.com.ar	3
	4	0	AgroMendoza S.A.	30712345701	Producción Agrícola	ventas@agromendoza.com.ar	4
	5	0	Distribuidora La Plata S.R.L.	30712345712	Comercio Mayorista	administracion@distiaplata.com.ar	5
	6	0	Turismo Costa Atlántica S.A.	30712345723	Servicios Turísticos	reservas@turismoatlantica.com.ar	6
	7	0	Alimentos del Norte S.A.	30712345734	Industria Alimenticia	calidad@alimentosnorte.com.ar	7
	8	0	FinanSalta S.R.L.	30712345745	Servicios Financieros	clientes@finansalta.com.ar	8
	9	1	Empresa Inactiva S.A.	30712345756	Actividad Cesada	NULL	NULL
	10	1	Sancor SA	12345678978	Industria alimenticia	sancor@empresa.com.ar	12
	11	1	empresa1	13245678966	empresa	empresa1@empresa.com	13
	12	1	empresa2	13245678977	empresa	empresa@empresa.com	14
	13	0	empresa3	12345678911	empresa	empresa3@empresa.com	15

Se verifica a través de MySQL Workbench con un consulta a la base de datos, que el registro fue ingresado exitosamente.

```
=== MENÚ PRINCIPAL ===
```

1. Crear Empresa
2. Leer empresa por Id
3. Leer empresa por CUIT
4. Buscar empresa por razon social
5. Listar empresas
6. Actualizar empresas
7. Eliminar empresa
8. Crear Domicilio Fiscal
9. Leer domicilio fiscal por Id
10. Listar todos los domicilios fiscales
11. Actualizar domicilio fiscal
12. Eliminar domicilio fiscal
0. Salir

Selecciona una opción: 4

Ingresa la razón social de la empresa: empresa3

Empresas encontradas:

```
Empresa{id=13, razonSocial='empresa3', cuit='12345678911', actividadPrincipal='empresa', email='empresa3@empresa.com', domicilioFiscal=DomicilioFiscal{id=15, calle='falsa', numero=123, ciudad='bs as', provincia='bs as', codigoPostal='1852', pais='Argentina'}}
```

Acá podemos ver otro ejemplo del uso del menú. Seleccionamos la opción número 4 para buscar una empresa a través de su razón social. Ingresamos por consola qué empresa queremos obtener y se nos muestra en pantalla.

Conclusiones del proyecto

Consideramos que el proyecto implementa adecuadamente la relación pedida en el informe e integra múltiples conceptos de arquitectura de software, programación orientada a objetos y base de datos relacionales.

A través del proyecto, logramos concretar los siguientes puntos:

Correcto funcionamiento: El programa concreta operaciones CRUD (crear, leer, actualizar y eliminar) satisfactoriamente, incluyendo eliminación de baja lógica y validaciones de entradas

Separación clara de capas (paquetes) del proyecto: El proyecto respeta la separación entre las capas **config**, **entities**, **dao** y **service**. El paquetes de DAO interactúa con la base de datos, mientras los servicios y entidades forman a nuestras clases y le atribuyen una lógica. Mientras que **config** es utilizada para nuestra conexión a la base de datos.

Consistencia en transacciones: Las operaciones son ejecutadas dentro un espacio de transacción utilizando setAutoCommit como false y rollback en caso de existir un error.

Mejoras futuras del proyecto

Pensamos que como futuras mejoras se podría reemplazar la interfaz de consola por un interfaz web usando Spring Boot por ejemplo, el cual es un framework que facilita la creación de aplicaciones empresariales con una mínima configuración. También podrían implementarse más validaciones como la validación del código postal según los datos proporcionados por usuario, validación del dominio proporcionado en el correo electrónico, entre otras.

Finalmente podrían agregarse test automáticos, especialmente si el proyecto se vuelve más grande y robusto.